

Databases :

Prepared By: Prof. Preeti Sharma

Ph.D. Pursuing CE/IT (LDCE, GTU, Ahmedabad)

M.E. IT (LDCE, Ahmedabad)

B.E. CE (VGEC, Chandkheda, Ahmedabad)

D.E. CE (GPG, Ahmedabad)

NoSQL Databases:

Introduction to NoSQL

- NoSQL stands for "Not Only SQL".
- NoSQL is an approach to database design that focuses on providing a mechanism for storage and retrieval of data that is modeled in means other than the tabular relations used in relational databases.
- NoSQL Database is a non-relational Data Management System that does not require a fixed schema.
- The traditional RDBMS (Relational Database System) can process only Structured types of data.
- But most of the data we get from the Internet may be Unstructured or Semi-Structured data, that can't be handled by RDBMS.
- So NoSQL Database came into the picture which is capable of handling all types of data. They provide a flexible, scalable, and high-performance solution for storing and managing large amounts of unstructured data.



What is NoSQL Database



High Scalability:

Horizontal Scaling and **Vertical Scaling** are two approaches for increasing the capacity and performance of a system to handle higher loads or more data.

Horizontal Scaling (Scaling Out):

Horizontal scaling involves adding more instances or nodes to a system to distribute the load. This approach expands capacity by increasing the number of machines or servers that work in parallel.

Key Features:

- **Distribution:** Load is distributed across multiple nodes.
 - **Fault Tolerance:** Enhances system reliability by spreading the risk of failure.
 - **Elasticity:** Allows for dynamic scaling in cloud environments, where nodes can be added or removed based on demand.
- 

Example: Consider a web application that experiences high traffic during peak times. To handle this traffic, you can horizontally scale by adding more web servers.

Scenario:

- **Initial Setup:** You start with a single web server that handles all incoming user requests.
- **Scaling Out:** As traffic increases, you add more web servers (e.g., three additional servers) behind a load balancer.
- **Load Balancer:** The load balancer distributes incoming requests across the four web servers, ensuring no single server is overwhelmed.

Vertical Scaling (Scaling Up):

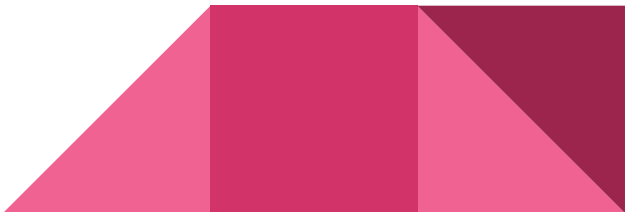
Definition: Vertical scaling involves increasing the capacity of a single server or machine by adding more resources such as CPU, memory, or storage. This approach enhances the power of an existing node to handle a higher load.

Key Features:

- **Resource Upgrade:** Adds more resources to the existing machine.
- **Simplicity:** Generally easier to implement since it involves upgrading a single machine.
- **Limited by Hardware:** Scaling is constrained by the maximum capacity of the machine.

Example: Consider a database server that handles a large number of queries and transactions. To improve performance, you can vertically scale the server by upgrading its hardware.

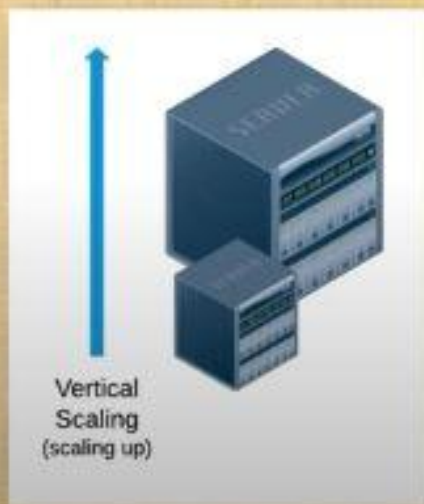
Scenario:

- **Initial Setup:** You start with a database server with 8 GB of RAM and 4 CPUs.
 - **Scaling Up:** As load increases, you upgrade the server to 32 GB of RAM and 16 CPUs.
 - **Result:** The upgraded server can handle more queries and transactions simultaneously.
- 

SQL vs NoSQL

Scalability

SQL



NoSQL



High Availability

High availability in NoSQL databases refers to the ability of the database to remain operational and accessible despite failures or issues within the system. This feature ensures that the database continues to provide its services even in the event of hardware failures, network issues, or other disruptions.

Example: Cassandra

Apache Cassandra is a popular NoSQL database known for its high availability features. Here's how it achieves high availability:

Data Replication: Cassandra replicates data across multiple nodes in a cluster. For example, if you configure a replication factor of 3, Cassandra will store three copies of each piece of data across different nodes. This means if one node fails, the data is still available from the other two nodes.



Decentralized Architecture: Cassandra uses a peer-to-peer architecture without a single point of failure. Every node in the cluster is equal and can handle read and write requests. This means that if one node fails, other nodes can take over its responsibilities without disrupting the service.

Automatic Failover: When a node fails, Cassandra automatically detects the failure and redirects requests to other nodes holding the same data. This ensures that the database continues to function smoothly without manual intervention.

Consistent Reads and Writes: Cassandra allows for configurable consistency levels, meaning you can choose how many replicas must acknowledge a read or write operation before it is considered successful. This flexibility

SQL vs NoSQL

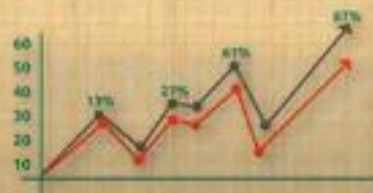
Data Storage Models

SQL

Customers Table		Widget Table		
Customer ID	Customer Name	Customer ID	Widget ID	Install Count
1	Customer 1	1	1	5
2	Customer 2	2	1	8
3	Customer 3	1	2	4
4	Customer 4	3	1	7
5	Customer 5	3	2	9
		1	3	7

Product Table			
Customer ID	Version	Quantity	Current Version
1	1.2	3	N
1	1.3	9	N
1	1.3	4	N
1	1.2	18	Y
2	1.2	8	Y
2	1.2	8	N
3	1.1	7	N
3	1.2	9	N
3	1.2	8	Y

NoSQL



Fast Performance:

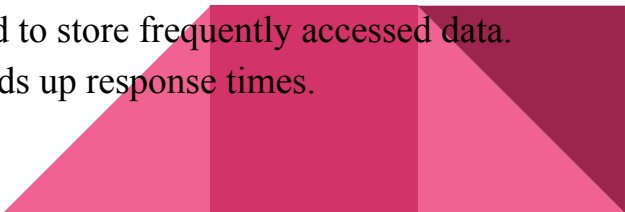
Sharding and Partitioning:

- **Sharding:** Distribute data across multiple servers or nodes to balance the load and increase throughput. Sharding can reduce the data volume handled by each node, thus improving performance.
- **Partitioning:** Similar to sharding, partitioning divides data into segments, but it's more about logical separation within a single node or server.

Replication:

- **Read Replicas:** Use read replicas to distribute read traffic and reduce the load on the primary node. This can improve read performance, especially for applications with high read-to-write ratios.

Caching:

- **In-Memory Caching:** Utilize in-memory caches like Redis or Memcached to store frequently accessed data. This reduces the need to query the database for common requests and speeds up response times.
- 

Optimized Queries:

- **Query Optimization:** Write efficient queries and avoid operations that require scanning large datasets. Make use of query execution plans and performance profiling tools provided by the database.

Data Compression:

- **Compression:** Apply data compression techniques to reduce the size of stored data and improve I/O performance. Many NoSQL databases support various compression methods.

Hardware and Infrastructure:

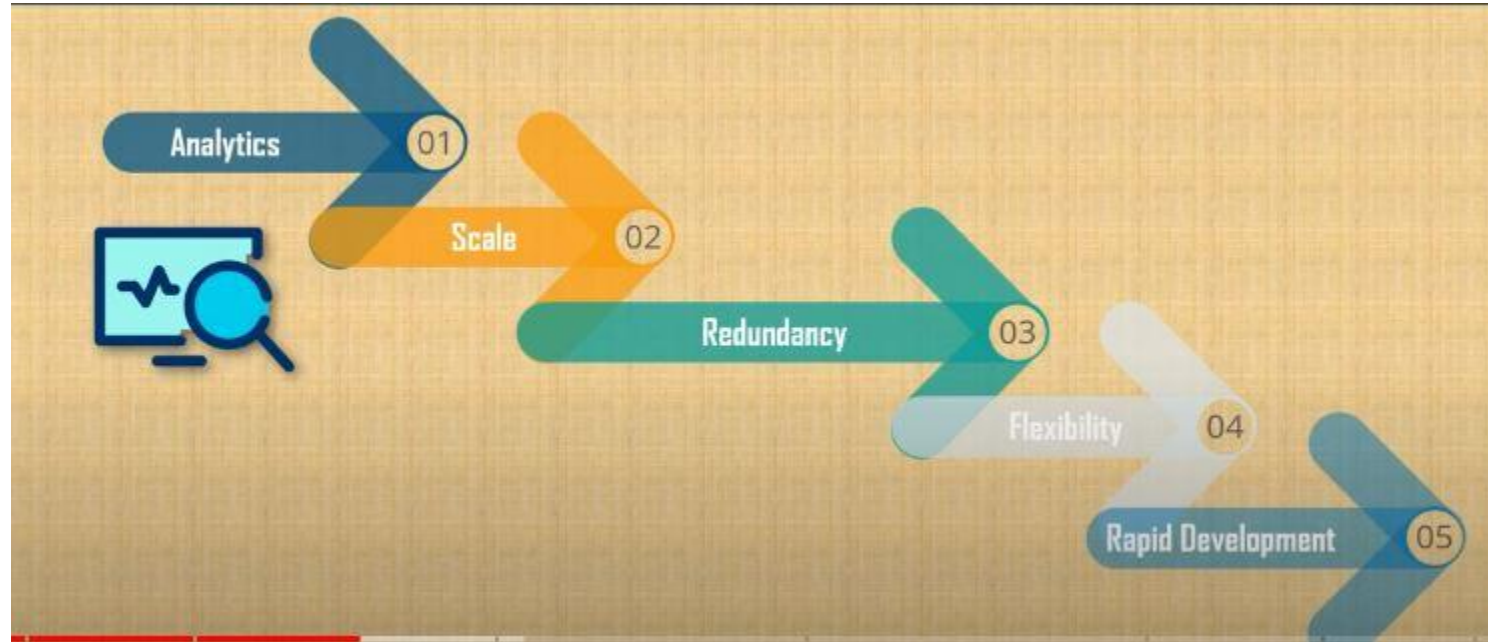
- **Resource Allocation:** Ensure that your database servers have adequate resources (CPU, RAM, disk I/O) to handle the expected workload.
- **Network Performance:** Optimize network performance to reduce latency and increase data transfer speeds between nodes.

Data Modelling:

Data Modeling for Access Patterns involves designing your data schema to efficiently support the most common queries and operations. This practice ensures that data retrieval is fast and minimizes the need for complex joins or aggregations.

Why NoSQL?

- Structure of different formats of data
- Created to handle big data
- Scaling the traffic



- **Flexibility:** A flexible schema in NoSQL databases refers to the ability to store and manage data without requiring a predefined, rigid schema structure. Unlike relational databases, where the schema must be defined before data can be inserted, NoSQL databases allow for more dynamic and adaptable data structures.
 - Schema Read
- **Schema read:**
 - No need to define a schema before storing data. Data can be stored in a raw or semi-structured format (e.g., JSON, XML, or key-value pairs).
 - The structure of the data is interpreted or enforced at the time of reading or querying. The schema is often specified in the query itself or inferred from the data as needed.
 - Example Use Cases: **Data Lakes (e.g., Hadoop):** Raw data can be stored in various formats, and the schema is applied at query time using tools like Hive

SOME OF THE REASONS TO USE NoSQL

Fast-paced Agile development

Storage of structured and semi-structured data

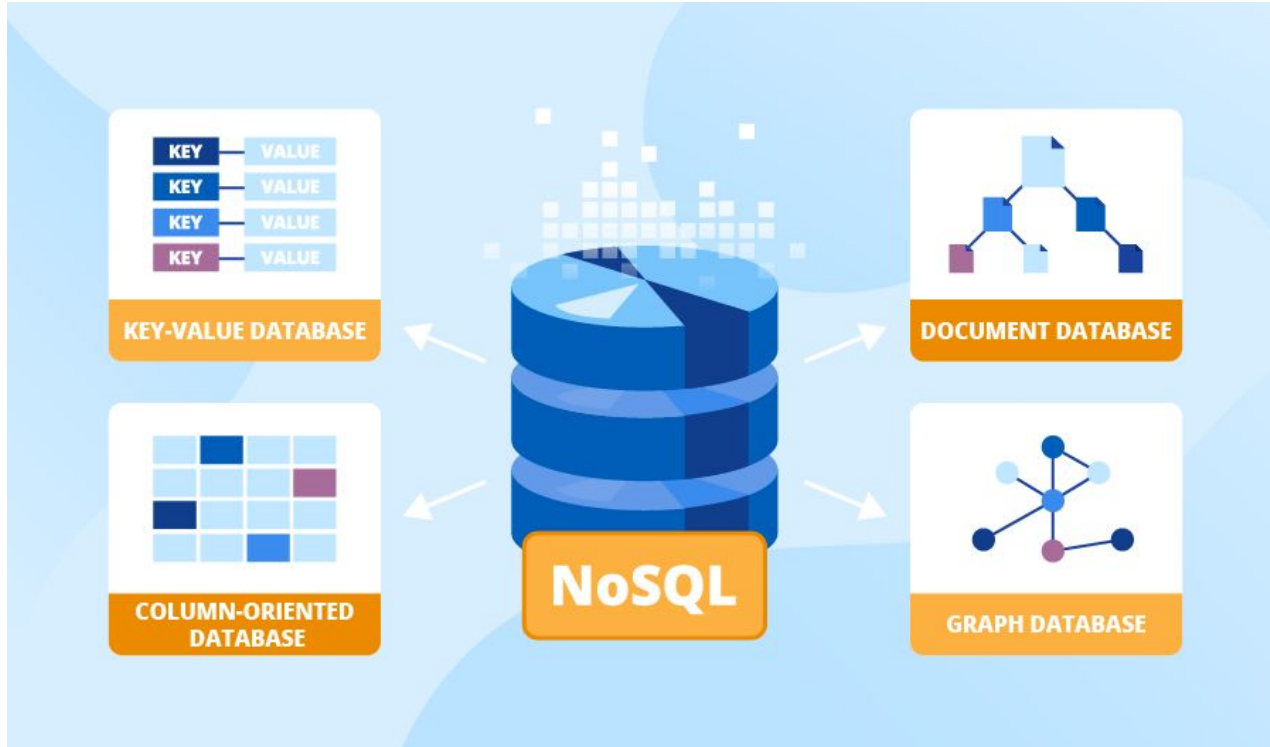
Huge volumes of data

Requirements for scale-out architecture

Modern application paradigms like microservices and real-time streaming

Types of NoSQL Databases:

Types of NoSQL Database:



1. Document Based Database:

1. Document Based Database:

- Document databases, also called document-oriented databases or a document store, are used to store and query **Semi-structured data**.
- The document-based database is a nonrelational database.
- Instead of storing the data in rows and columns (tables), it uses the documents to store the data in the database.
- Data is stored in a JSON-like document similar to the data objects that developers use in application code, making it easier to create and update applications without referencing a primary schema.
- Document-based databases store data in formats like JSON, BSON, or XML, which can be highly flexible and accommodating for **Unstructured data**.
- In the Document database, the particular elements can be accessed by using the index value that is assigned for faster querying.



Example of Document based Database:

Beers Table

1167	Ale C	Miller	570
3424	Beerio	Ians	340
5612	Amstel	Amtel	121
2409	Colt's	BeerCo	98

Beers Documents



How Foreign key concept works in Document databases?

In document-based databases, the concept of foreign keys—commonly used in relational databases to enforce relationships between tables—is handled differently due to the schema-less and flexible nature of document databases.

METHOD -1: Embedding Documents

Concept: In document-based databases, instead of using foreign keys, you often embed related data directly within a single document. This approach is suitable when the related data is tightly coupled and often accessed together.

Example: If you have a **User** document and each user has a list of **Orders**, you can embed the **Orders** directly within the **User** document.



Example:

```
{
  "user_id": "12345",
  "name": "John Doe",
  "orders": [
    {
      "order_id": "abc",
      "product": "Laptop",
      "price": 1200
    },
    {
      "order_id": "def",
      "product": "Mouse",
      "price": 25
    }
  ]
}
```

Benefits:

- Reduces the need for joins, which can improve performance.
- Simplifies data retrieval by keeping related data together.

Limitations:

- Embedding can lead to document bloat if the embedded data grows large.
- Updates to the embedded data might be more complex if multiple documents need to be updated.

METHOD - 2: Referencing Documents

Concept: Instead of embedding, you can use references (or pointers) to link documents. Each document contains an identifier (like a `user_id` or `order_id`) that refers to another document.

Example: You have a `User` document and an `Order` document stored separately. The `Order` document contains a reference to the `User`.

User Document:

```
{  
  "user_id": "12345",  
  "name": "John Doe"  
}
```

Order Document:

```
{  
  "order_id": "abc",  
  "user_id": "12345",  
  "product": "Laptop",  
  "price": 1200  
}
```

Benefits:

- Reduces document size by avoiding data duplication.
- Allows more flexible data management and scalability.

Limitations:

- Requires additional queries or application logic to join or fetch related data.
 - Can be less performant for complex queries compared to embedding, especially if multiple lookups are needed.
- 

METHOD - 3: Using Database Features

Some document-based databases provide built-in features to handle relationships more efficiently:

- **MongoDB:** Offers aggregation frameworks that can perform lookups and joins between collections using the `$lookup` operator.
- **Couchbase:** Supports N1QL (SQL for JSON) queries, which can handle joins and querying across documents.

Choosing the Right Approach:

The choice between embedding and referencing depends on your application's requirements:

- **Embedding:** Use when related data is frequently accessed together and document size is manageable.
 - **Referencing:** Use when related data is accessed separately or can grow large, making embedding impractical.
- 

Different Document based NoSQL Databases:

1. MongoDB
2. Couchbase
3. RavenDB
4. Amazon DocumentDB
5. MarkLogic



Use cases and Applications of Document Based Databases:

1. Content Management Systems (CMS):

Company: The New York Times

- **Use Case:** The New York Times uses **MongoDB** to manage its vast array of digital content, including articles, multimedia files, and metadata. MongoDB's document-oriented storage is well-suited for managing complex and nested content structures.
- **Benefits:** MongoDB allows for rapid content updates and efficient querying of large volumes of media and articles.

2. IoT (Internet of Things):

Company: Bosch

- **Use Case:** Bosch utilizes **MongoDB** to handle data from various IoT sensors and devices, including device performance metrics and environmental data. MongoDB's ability to handle high-velocity data is crucial for real-time analytics.
- **Benefits:** MongoDB's horizontal scaling and real-time processing capabilities are essential for managing and analyzing large volumes of sensor data.

3. Real-Time Data Processing:

Company: Paypal

Use Case: Real-time analytics are critical for detecting fraudulent transactions, monitoring payment trends, and providing immediate feedback to users. PayPal needs to process data in real time to make instant decisions and improve the user experience.

Couchbase Features:

- **Low-Latency Operations:** Couchbase's architecture is designed to provide low-latency data access, which is essential for real-time analytics and decision-making.
- **N1QL Query Language:** Couchbase's N1QL query language allows for SQL-like queries on JSON documents. This capability enables complex real-time analytics queries to be executed efficiently.

2. Key-Value NoSQL Database:

2. Key-Value NoSQL Database:

- Key-value stores use a hash table that stores a pointer, which is the key, which then points to a value that stores some form of information or data.
- Key-value databases handle **Semi-structured data** effectively. The value in a key-value pair can be a semi-structured format like JSON, XML, or other document-like structures. This allows flexibility in storing data with nested or hierarchical properties.

Example: Storing a JSON document as a value, e.g.,

"user:123": //Key: This is a unique identifier used to access the value stored in the database.

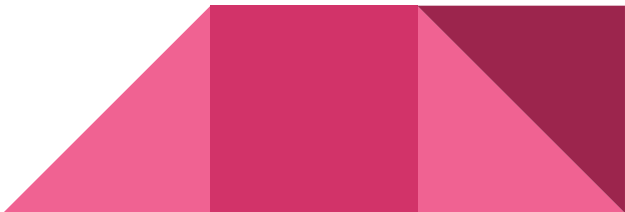
{

 "name": "John",

 "age": 30,

 "address": { "city": "New York", "state": "NY" }

}



Example of Key Value Database:

Storing Social Media Posts in a Key-Value Database:

Basic Storage Approach

- **Key:** A unique identifier for each post, such as a post ID.
- **Value:** The content of the post, which could be text, images, videos, and metadata (like timestamps, user IDs, or tags).

Example:

Key: "post:12345"

Value:

```
{  
  "user_id": "67890",  
  "timestamp": "2024-07-31T12:34:56Z",  
  "content": "This is a social media post!",  
  "media": ["image1.jpg", "video1.mp4"],  
  "tags": ["example", "post"]  
}
```

Store Metadata and References in the Key-Value Database:

- Instead of storing the actual media files directly in the key-value database, you store metadata and references to the media files. This keeps the key-value database lean and focused on fast lookups while using a separate storage service for the actual media content.

Example:

Key: "post:12345"

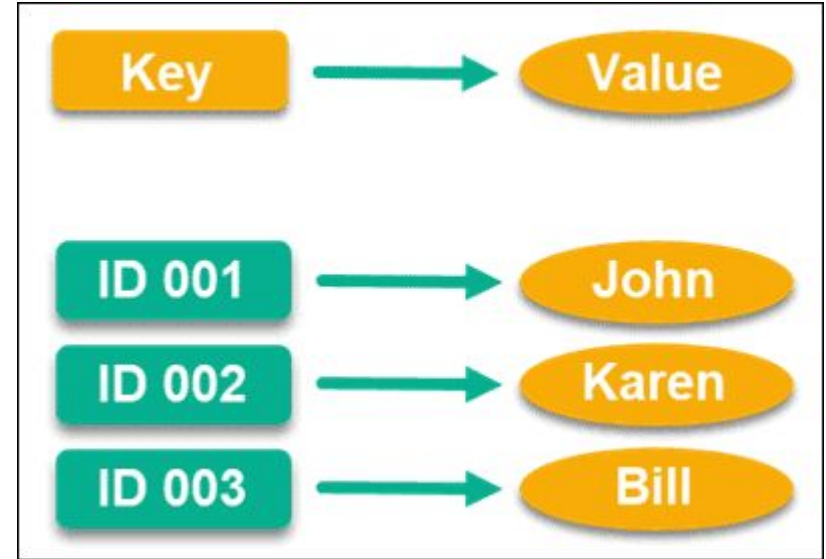
Value:

```
{
  "user_id": "67890",
  "timestamp": "2024-07-31T12:34:56Z",
  "content": "This is a social media post!",
  "media": [
    {
      "type": "image",
      "path": "images/post_12345_image1.jpg",
      "url": "https://example.com/images/post\_12345\_image1.jpg"
    },
    {
      "type": "video",
      "path": "videos/post_12345_video1.mp4",
      "url": "https://example.com/videos/post\_12345\_video1.mp4"
    }
  ],
  "tags": ["example", "post"]
}
```


Use Cases and Applications of Key Value Database:

Use Cases

- **Session Management:** Storing user session data where each session ID is a key, and session data is the value.
- **Caching:** Speeding up data retrieval by caching frequently accessed data.
- **User Profiles:** Storing user profile information where user IDs are the keys and profile data is the value.



Different Key Value NoSQL Databases:

1. Redis:

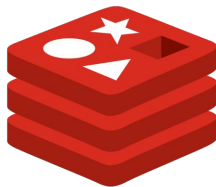
- An in-memory key-value store known for its speed and support for various data structures like strings, hashes, lists, sets, and sorted sets.
- Redis provides fast read and write operations, making it ideal for caching results of expensive queries or computations.
 - **Example:** Caching web page content, API responses, or database query results

Storing Data:

- Redis provides simple commands for setting and retrieving data. For example, you can use **SET** to store a key-value pair and **GET** to retrieve it.

SET mykey "some value"

GET mykey



redis

Expiration and Eviction

- **Time-to-Live (TTL):** Redis allows you to set an expiration time on cached data using the EXPIRE or SETEX commands. After the TTL expires, the key-value pair is automatically removed from Redis.

SETEX mykey 3600 "some value" # Expires in 3600 seconds (1 hour)

Replication and High Availability

- **Replication:** Redis supports master-slave replication, where a master Redis instance can replicate data to one or more slave instances. This setup can be used to distribute read queries and provide redundancy.



Use Case: Web Application Caching

A high-traffic e-commerce website needs to cache product details to reduce database load and speed up page load times.

Implementation:

- **Cache Product Data:** Store product details (e.g., name, price, description) in Redis. When a user requests product information, the application first checks Redis.
- **Fallback:** If the product data is not found in the cache (a cache miss), the application fetches it from the primary database, stores it in Redis for future requests, and then returns it to the user.
- **Expiration:** Set an expiration time on the cached data to ensure it is refreshed periodically and to avoid stale data.

- **Riak KV (Key-Value):** Riak KV is designed with a focus on high availability and fault tolerance, using techniques like data replication and automatic failover to ensure that your data remains accessible even if some nodes fail.
 - **Example:** E-commerce platforms where uptime is critical, and the system must handle failures without affecting user experience.
- **Amazon DynamoDB:** DynamoDB provides single-digit millisecond response times and can handle millions of requests per second, making it ideal for high-performance applications.
 - **Example:** Gaming leaderboards, real-time recommendation engines, and ad tech platforms.
- **Couchbase:** Couchbase's in-memory architecture and indexing capabilities ensure fast data access and querying. It supports high-speed operations essential for real-time analytics and big data applications.
 - **Example:** Real-time analytics platforms, financial trading systems, and IoT data processing.

2. Amazon DynamoDB:



A fully managed NoSQL database service provided by AWS, known for its scalability and performance.

Use Case:

Social media applications that require managing user-generated content, profiles, and social interactions.

Implementation:

Tables:

- **Table Creation:** Data in DynamoDB is organized into tables. Each table is defined with a primary key, and you can also define secondary indexes to support additional query patterns.
- **Primary Key:** The primary key can be a simple primary key (a single attribute) or a composite primary key (a partition key and a sort key). This key uniquely identifies each item in the table.

Items:

- **Data Representation:** Data in DynamoDB is stored as items. An item is a collection of attributes, where each attribute is a key-value pair.
- **Attributes:** Attributes can be of various types, including scalar types (e.g., string, number, binary), document types (e.g., lists, maps), and sets (e.g., string sets, number sets).

Example: Posts and Content

- **Table Name:** Posts
- **Primary Key:** PostID (Partition Key)
- **Attributes:** UserID, Content, Timestamp, LikesCount, CommentsCount, MediaURL, etc.

Implementation:

- **Post Storage:** Each post is an item with a unique PostID, storing the post content and associated metadata.
- **Access Patterns:** To fetch posts by a user or retrieve recent posts, use a Global Secondary Index (GSI) with UserID and Timestamp as keys.

```
{"PostID": "post456",  
  "UserID": "user123",  
  "Content": "Just had a great lunch!",  
  "Timestamp": "2024-08-03T14:30:00Z",  
  "LikesCount": 45,  
  "CommentsCount": 10,  
  "MediaURL": "https://example.com/image.jpg" }
```

Followers and Following

Table Name: Followers

- **Primary Key:** UserID (Partition Key), FollowerID (Sort Key)

Example Item:

```
{
  "UserID": "user123",
  "FollowerID": "user456",
  "Timestamp": "2024-08-03T15:00:00Z"
}
```

Likes and Reactions

Table Name: Likes

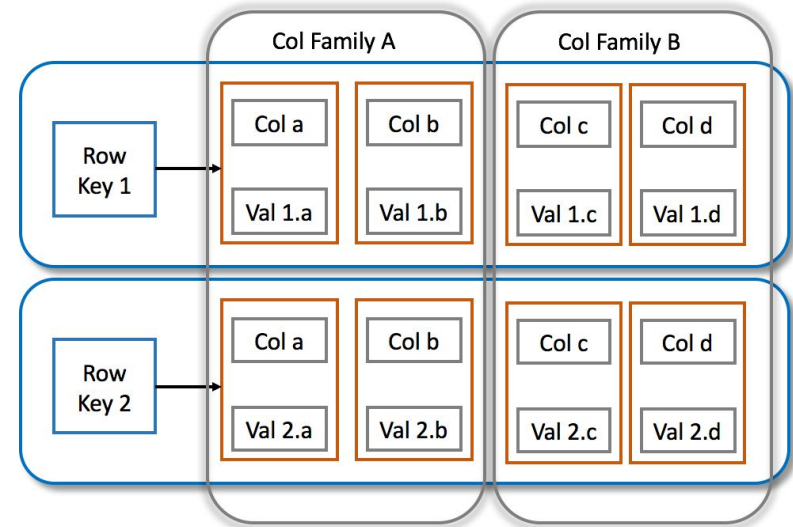
- **Primary Key:** PostID (Partition Key), UserID (Sort Key)

```
{
  "PostID": "post456",
  "UserID": "user789",
  "Timestamp": "2024-08-03T15:05:00Z"
}
```


3. Column Family Stores NoSQL Database:

3. Column Family Store NoSQL Database:

- Column-family NoSQL databases are a type of distributed NoSQL database designed to handle large volumes of data with high performance and scalability.
- They are particularly well-suited for applications that require fast read and write operations on vast datasets.
- The data model of column-family databases is inspired by the way data is organized and stored in relational databases but with a more flexible schema and improved scalability.



Different Column Family Store Databases:

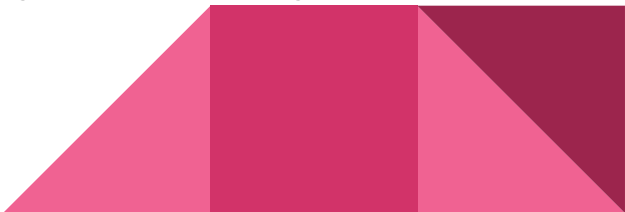
1. Apache Cassandra:

- **Description:** A highly scalable, distributed database designed for high availability and fault tolerance. It uses a column-family data model and is known for its ability to handle large amounts of data across many servers without a single point of failure.
- **Use Cases:** Real-time analytics, recommendation systems, time-series data, and IoT applications.

2. HBase:

- **Description:** A distributed, scalable column-family database that runs on top of the Hadoop Distributed File System (HDFS). It is designed to handle large-scale data storage and retrieval with low latency.
- **Use Cases:** Big data applications, real-time data processing, and large-scale data warehousing.

3. ScyllaDB:

- **Description:** A high-performance, distributed NoSQL database that is compatible with Apache Cassandra but aims to provide better performance and lower latencies.
 - **Use Cases:** High-throughput applications, real-time analytics, and large-scale data storage.
- 

Use Cases and Applications of Column Family Store:

1. Time-Series Data:

Company: Netflix

- **Use Case:** Netflix uses column-family databases to handle time-series data for monitoring and analyzing user activities and streaming metrics. This includes tracking user viewing patterns, stream quality, and service performance.
- **Technology:** Netflix utilizes **Apache Cassandra** to manage their vast amount of time-series data efficiently, allowing them to perform real-time analytics and ensure a smooth streaming experience for users.

2. Financial Services:

Company: eBay

- **Use Case:** eBay uses column-family databases to manage and process data related to transactions, user activities, and inventory. They need a scalable solution to handle high transaction volumes and provide real-time search capabilities.
- **Technology:** eBay has utilized **Apache HBase** to handle their large-scale transactional data and provide efficient read/write operations.

3. E-Commerce:

Company: Walmart

- **Use Case:** Walmart uses column-family databases to handle vast amounts of data related to inventory management, customer transactions, and product recommendations. This is crucial for managing their large-scale e-commerce platform and brick-and-mortar operations.
- **Technology:** Walmart has used **Apache HBase** for managing and processing their large datasets, providing scalable and efficient data handling for their retail operations.

4. Search and Recommendation Systems:

Company: Yahoo

- **Use Case:** Yahoo employs column-family databases to support their search engine and recommendation systems. This involves managing large volumes of search queries, user preferences, and content indexing.
- **Technology:** Yahoo has used **Apache HBase** for their search and recommendation infrastructure, leveraging its scalability and performance capabilities to handle large-scale

4. Graph based NoSQL Database:

4. Graph based NoSQL Database:

- Graph-based NoSQL databases are designed to efficiently manage and query complex relationships between data points.

Core Concepts

1. Nodes (Vertices):

- **Definition:** Represent entities or objects in the graph.
- **Example:** In a social network, nodes could represent users, posts, or comments.

2. Edges (Relationships):

- **Definition:** Represent the connections between nodes. Edges can be directed or undirected and may have properties.
- **Example:** In a social network, edges might represent "friends with," "likes," or "follows."

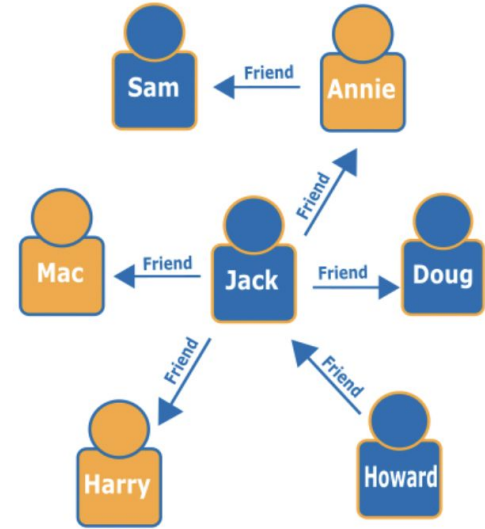
3. Properties:

- **Definition:** Key-value pairs associated with nodes and edges. Properties provide additional context or attributes.
 - **Example:** A user node might have properties such as "name" and "email," while a "likes" edge might have a property like "date."
- 

Use Cases and Applications of Graph based Databases:

1. Finding Mutual Friends:

- The following graph shows an example of a social network graph. Given the people (nodes) and their relationships (edges), you can find out who the “friends of friends” of a particular person are—for example, the friends of Howard’s friends.



2. Fraud Detection:

Company: Paypal

Usecase: PayPal uses graph databases to eliminate fraud by analyzing their user data from three perspectives:

1. **Asset Sharing** - When two accounts share the same attributes (home address, credit card number, social security number, etc.) then PayPal can place an edge linking them in the database. Using this, they can quickly identify abnormal behaviors. If 50 accounts all share the same 2 bedroom apartment as their home address, then that's *probably* worth investigating.
2. **Transaction Patterns** - When two users transact with each other, that is stored as an edge in the graph database. PayPal is able to quickly analyze all their transactions and search for strange behaviors. A common pattern that's typically flagged for fraud is the "ABABA" pattern where users A and B will repeatedly send money back and forth between each other in a very short period.
3. **Graph Features** - The structural characteristics of the graph (connected communities of accounts, vertices that have lots of connections, degree of clustering amongst nodes, etc.) are very useful for predicting potential fraud. For example, if you have a dense cluster of 10 accounts (*these 10 accounts have a lot of edges connecting them all*) where 5 are identified as fraudsters, you might want to pay extra attention to the remaining 5 accounts.

3. Recommendation engines:

- Graph databases are a good choice for recommendation applications.
- With graph databases, you can store in a graph relationships between information categories such as customer interests, friends, and purchase history.
- You can use a highly available graph database to make product recommendations to a user based on which products are purchased by others who follow the same sport and have similar purchase history.
- Or, you can identify people who have a friend in common but don't yet know each other, and then make a friendship recommendation.

Different Graph based NoSQL Databases:

1. Neo4j



2. Amazon Neptune



Amazon Neptune

3. OrientDB



4. GraphDB



GraphDB

Example: Neo4J

Insert users and their friendships into the graph database. Here's an example using Cypher, the query language for Neo4j, a popular graph database:

Add Users:

```
CREATE (a:User {user_id: '1', name: 'Alice'})  
CREATE (b:User {user_id: '2', name: 'Bob'})  
CREATE (c:User {user_id: '3', name: 'Charlie'})
```

Add Friendships:

```
MATCH (a:User {user_id: '1'}), (b:User {user_id: '2'})  
CREATE (a)-[:FRIEND]->(b)
```

```
MATCH (a:User {user_id: '1'}), (c:User {user_id: '3'})  
CREATE (a)-[:FRIEND]->(c)
```

```
MATCH (b:User {user_id: '2'}), (c:User {user_id: '3'})  
CREATE (b)-[:FRIEND]->(c)
```

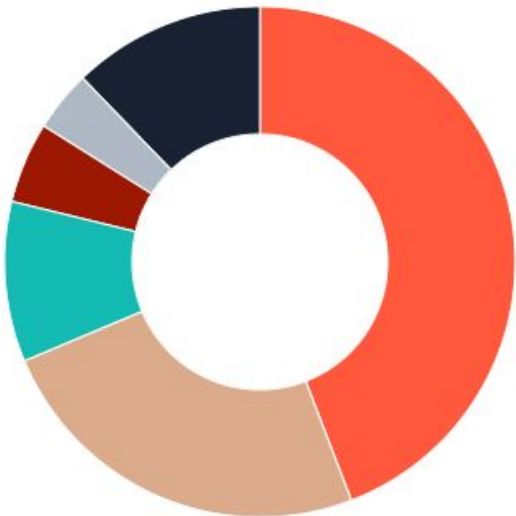




Statistics of NoSQL Databases:

Top 5 NoSQL Databases:

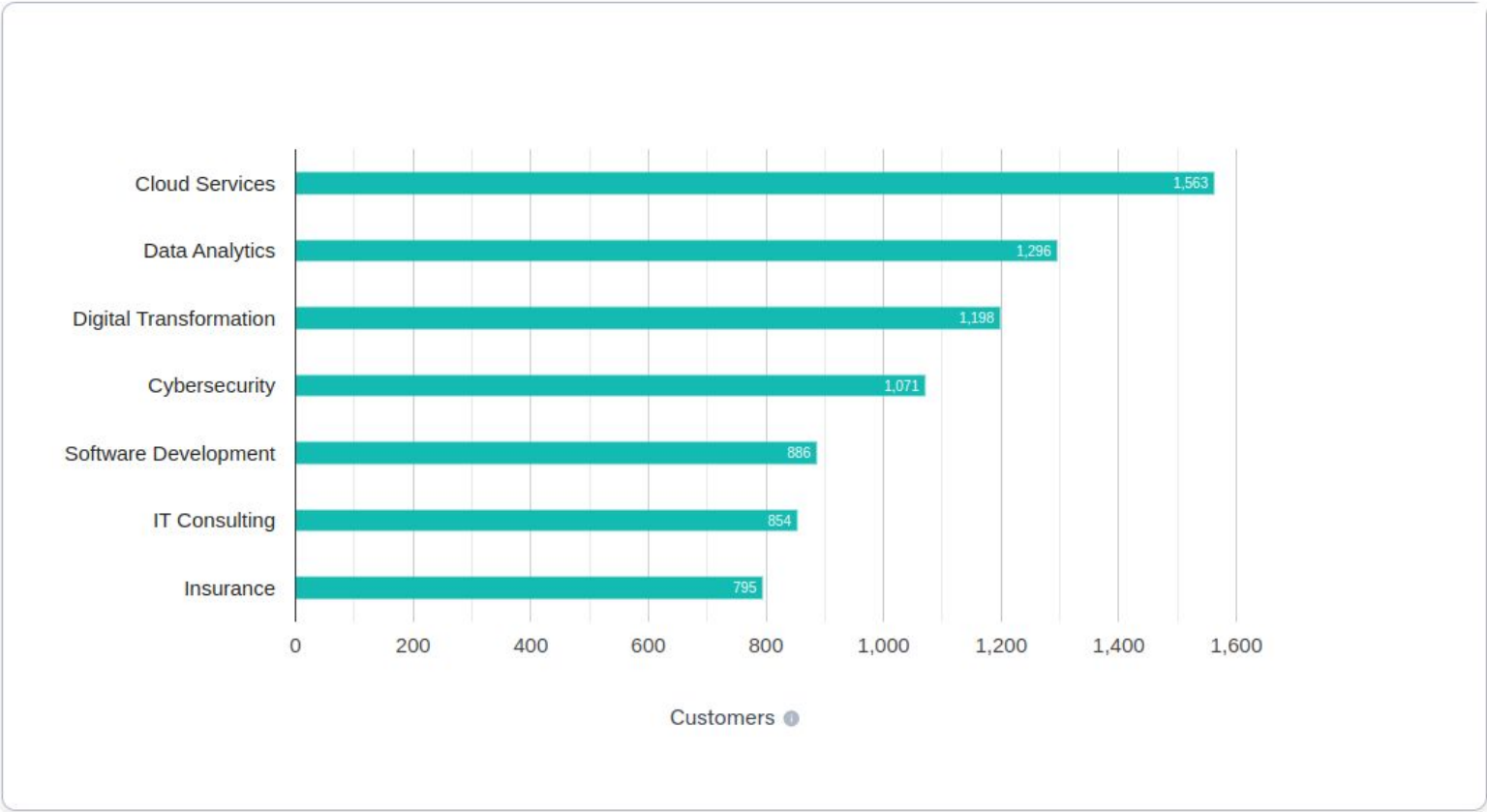
Over 108,140 companies are using NoSQL Databases tools. MongoDB with 44.17% market share (47,763 customers), NoSQL with 24.49% market share (26,480 customers), Amazon DynamoDB with 10.15% market share (10,975 customers).



Technology	Market Share (Est.) ⓘ	Customers ⓘ
MongoDB	44.17%	47,763
NoSQL	24.49%	26,480
Amazon DynamoDB	10.15%	10,975
Apache Cassandra	5.13%	5,549
Hbase	3.86%	4,173
Others	12.21%	13,200

NoSQL Databases Customers by Industry

Top industries that use NoSQL Databases for Cloud Services (1,563), Data Analytics (1,296), Digital Transformation (1,198).



Properties and Theorem:

SQL Properties - ACID:

What are ACID and BASE properties?

- ACID and BASE are two sets of principles related to database systems that describe how they handle transactions and consistency.

ACID Properties:

- ACID is a set of properties that ensures reliable processing of database transactions. These properties are crucial for maintaining data integrity and consistency in traditional relational databases.

Atomicity: A transaction is treated as a single, indivisible unit of work. It either completes in its entirety or not at all. Partial completions are not allowed.

- **Example:** If a transaction involves transferring money from one account to another, both the debit and credit operations must succeed or fail together.
- 

Consistency: A transaction must transform the database from one consistent state to another. This means that all data rules and constraints are maintained throughout the transaction.

- **Example:** If a transaction updates a bank account balance, it must adhere to rules like account balance constraints.

Isolation: Transactions should not interfere with each other. Intermediate states of a transaction should not be visible to other transactions until the transaction is completed.

- **Example:** If two transactions are simultaneously updating different accounts, their operations should not affect each other's intermediate results.

Durability: Once a transaction is committed, its effects are permanently stored in the database, even in the event of a system failure.

- **Example:** After a transaction has been successfully processed, the changes should persist even if there's a crash or power failure.
- 

NoSQL Properties - BASE:

BASE Properties

BASE is an acronym that represents an alternative set of principles, often associated with NoSQL databases. BASE is more flexible and is suited for scenarios where strict ACID properties may not be necessary or practical due to scalability and performance requirements.

1. **Basically Available:** The system guarantees that data is available for reading and writing, but it may not be up-to-date. The system will respond to requests even if it's not fully synchronized.
 - **Example:** In a distributed NoSQL database, some nodes may have stale data, but the system remains operational and responsive.
2. **Soft State:** The state of the system may change over time, even without new input. This is due to the eventual consistency model where updates are propagated asynchronously.
 - **Example:** Data updates might take some time to propagate through all nodes, leading to temporary inconsistencies.



3. Eventually Consistent:

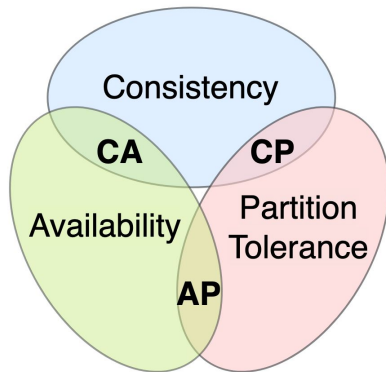
- **Definition:** The system guarantees that, given enough time, all replicas of data will converge to the same state. This means that, while data may be temporarily inconsistent, it will eventually become consistent.
- **Example:** In a distributed system like Cassandra, updates may be seen at different times on different nodes, but all nodes will eventually reflect the same data.



CAP Theorem:

CAP Theorem:

- The CAP Theorem, also known as **Brewer's Theorem**, is a fundamental principle in distributed systems that describes the trade-offs between three key properties: **Consistency, Availability, and Partition Tolerance**.
- Formulated by computer scientist Eric Brewer in 2000 and later formalized by Seth Gilbert and Nancy Lynch, the theorem states that it is impossible for a distributed system to simultaneously achieve all three of these properties.



CAP Theorem Components:

Consistency:

- **Definition:** Every read operation returns the most recent write result, meaning that all nodes in the system see the same data at the same time. This ensures that once a write is acknowledged, all subsequent reads will reflect that write.
- **Example:** In a consistent system, if a user updates their profile information, all users querying the profile will see the updated information immediately.

Availability:

- **Definition:** Every request (read or write) receives a response, whether successful or not. The system is always operational and can process requests even if some nodes are down.
- **Example:** An available system will always return a response to a user query, even if it cannot guarantee that the response is the most recent or consistent.

Partition Tolerance:

- **Definition:** The system continues to operate and maintain its properties even if there is a network partition or communication breakdown between nodes. The system can handle failures in communication without losing data or becoming unusable.
- **Example:** A partition-tolerant system can still function and process requests even if some parts of the network are temporarily disconnected.

Theorem Statement:

According to the CAP Theorem, a distributed system can achieve at most two out of the three properties at any given time:

1. **Consistency and Availability (CA):**

- The system guarantees that all nodes have the same data (Consistency) and every request receives a response (Availability). However, it cannot handle network partitions effectively.
- **Example:** Traditional relational databases often prioritize CA but may struggle to maintain this in the face of network partitions.

2. **Consistency and Partition Tolerance (CP):**

- The system guarantees that all nodes have the same data (Consistency) and can handle network partitions (Partition Tolerance). However, it may sacrifice Availability during partitions, meaning some requests may not be processed.
- **Example:** Systems like HBase or MongoDB in certain configurations may focus on CP, ensuring data consistency and partition tolerance but potentially being unavailable during network issues.

3. **Availability and Partition Tolerance (AP):**

- The system guarantees that every request receives a response (Availability) and can handle network partitions (Partition Tolerance). However, it may not always provide consistent data across all nodes.
- **Example:** NoSQL databases like Cassandra or Couchbase often prioritize AP, ensuring they remain operational and responsive even if some nodes are out of sync.

Why only 2 properties can be achieved?

- The reason why we can't use all 3 is because of the nature of properties, for eg if there is a partition between nodes, it wouldn't be possible to maintain consistency across all nodes as write will only happen in node 1 but not in node 2 due to partition.

Let's understand all the possible combinations among the 3 which can be used.

- **CA:-** If your aim is a highly consistent & available system then you choose these properties but you will need to sacrifice the partition tolerance, and in this case, if a network partition occurs, your system might become unavailable or operate in a limited capacity as it will need to ensure data consistency.

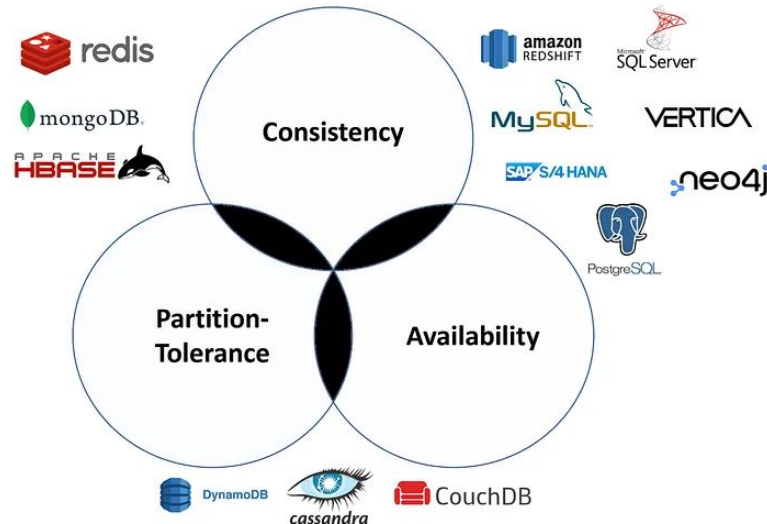
***Example:-** In e-commerce, maintaining a stock level consistent throughout for all users is an important task. In banking systems, it is must that data is consistent for users.*



CP:- If choosing consistency and partition tolerance then you are aiming for a highly consistent system even during network partition, but this will result in low availability as some of your nodes may not be available when partitioning occurs.

AP:- You are looking for a highly available system even during the network partition then you go for Availability & Partitioning. However, you will need to compromise on the consistency part as data may not be the same across all nodes during network partitions.

Example:- On a social media platform, it becomes important that the system is accessible every time, user can be shown outdated data in case of network partitioning.



NewSQL Database:

NewSQL Database:

- The concept of **NewSQL** represents an effort to blend the strengths of both traditional SQL databases and NoSQL databases, thereby offering a modern database solution that combines the best of both worlds.

$$\text{SQL} + \text{NoSQL} = \text{NewSQL}$$

- NewSQL databases aim to address the limitations of traditional relational databases in handling large-scale, high-throughput workloads, while retaining the benefits of SQL-based querying and ACID (Atomicity, Consistency, Isolation, Durability) transactions.



Key Characteristics of NewSQL Databases:

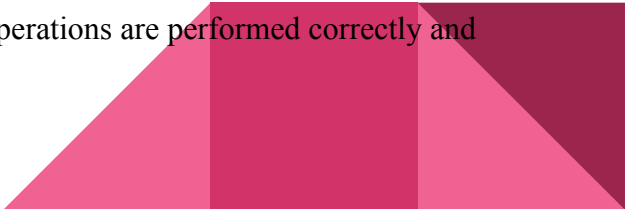
Scalability:

- **Horizontal Scalability:** NewSQL databases are designed to scale out horizontally by distributing data across multiple nodes or servers. This allows them to handle large volumes of data and high transaction rates efficiently.
- **Elasticity:** Many NewSQL databases support dynamic scaling, where resources can be added or removed based on demand.

SQL Compatibility:

- **SQL Query Language:** NewSQL databases use SQL as their query language, making them compatible with existing SQL tools and skills. This allows users to leverage familiar SQL features such as complex queries, joins, and aggregations.
- **Existing Ecosystem:** Users can utilize established SQL tools for development, management, and analytics.

ACID Transactions:

- **Consistency:** NewSQL databases maintain strong consistency and adhere to ACID properties, ensuring reliable transaction processing and data integrity.
 - **Transactional Integrity:** They provide robust support for transactions, ensuring that operations are performed correctly and completely.
- 

Performance:

- **Optimized for Modern Hardware:** NewSQL databases are designed to take advantage of modern hardware architectures, such as multi-core processors and distributed systems, to deliver high performance and low latency.
- **Advanced Indexing and Caching:** They often include features like advanced indexing, in-memory processing, and caching to enhance query performance.

High Availability:

- **Fault Tolerance:** Many NewSQL databases offer built-in replication and failover mechanisms to ensure high availability and fault tolerance.
- **Automatic Recovery:** They provide automatic recovery from failures to minimize downtime and maintain system reliability.

SQL vs. NoSQL: A Quick Comparison

Traditional SQL Databases:

- **Data Model:** Relational, based on structured tables with rows and columns.
- **Query Language:** SQL (Structured Query Language) for querying and managing data.
- **ACID Transactions:** Strong guarantees of Atomicity, Consistency, Isolation, and Durability.
- **Scalability:** Typically scales vertically by adding resources to a single server.
- **Schema:** Fixed schema with predefined tables and relationships.

NoSQL Databases:

- **Data Model:** Flexible models such as key-value, document, column-family, or graph.
 - **Query Language:** Varies by database; may not support standard SQL queries.
 - **Eventual Consistency:** Often prioritize availability and partition tolerance over strong consistency (as per the CAP theorem).
 - **Scalability:** Scales horizontally by distributing data across multiple servers or nodes.
 - **Schema:** Schema-less or flexible schema, allowing dynamic changes to the data structure.
- 

SQL vs NoSQL:

SQL	MongoDB	DynamoDB	Cassandra	Couchbase
Table	Collection	Table	Table	Data bucket
Row	Document	Item	Row	Document
Column	Field	Attribute	Column	Field
Primary key	ObjectId	Primary key	Primary key	Document ID
Index	Index	Secondary index	Index	Index
View	View	Global secondary index	Materialized view	View
Nested table or object	Embedded document	Map	Map	Map
Array	Array	List	List	List



NewSQL: The Synthesis of SQL and NoSQL

1. Relational Data Model and SQL Compatibility:

- **SQL-like Queries:** NewSQL databases use SQL or SQL-like query languages to allow for complex queries, joins, and aggregations, similar to traditional relational databases.
- **Schema:** They often maintain a structured schema, providing strong data integrity and consistency.

2. Horizontal Scalability:

- **Distributed Architecture:** NewSQL databases are designed to scale horizontally, distributing data and workload across multiple servers or nodes. This is a feature borrowed from NoSQL databases, which helps handle larger datasets and high-throughput applications.
 - **Data Distribution:** Data is partitioned and distributed across nodes, allowing the system to handle increased loads and provide high availability.
- 

3. ACID Transactions and Consistency:

- **Strong Consistency:** NewSQL databases provide ACID guarantees, ensuring that transactions are processed reliably and consistently, similar to traditional SQL databases. This is a key differentiator from many NoSQL databases that may offer eventual consistency.

4. Performance Enhancements:

- **In-Memory Processing:** Many NewSQL databases use in-memory processing and advanced indexing techniques to achieve high performance and low latency, addressing one of the key advantages of NoSQL databases.
 - **Optimized for Modern Workloads:** They are designed to efficiently handle modern application workloads, including real-time analytics and high-velocity transactions.
- 

Real-World Examples of NewSQL Databases:

1. **Google Spanner:** A globally distributed database that combines SQL querying with horizontal scalability and strong consistency. It uses a distributed architecture and implements a global consensus protocol for transaction consistency.
2. **CockroachDB:** A distributed SQL database designed to scale horizontally while maintaining strong consistency and ACID properties. It offers SQL support and is optimized for fault tolerance and high availability.
3. **NuoDB:** A distributed SQL database that provides scalable and elastic cloud-native solutions while supporting SQL queries and ACID transactions.
4. **VoltDB:** An in-memory, distributed SQL database optimized for high-throughput and low-latency transactions. It supports SQL and provides strong consistency guarantees.



SQL Characteristics in NewSQL:

1. Relational Data Model:

- **Example: Google Spanner**

- **Description:** Google Spanner uses a relational data model with SQL-like querying. It supports complex queries, joins, and transactions with strong consistency, just like traditional SQL databases. Spanner's schema is structured, allowing it to maintain data integrity through ACID transactions.

2. ACID Transactions:

- **Example: CockroachDB**

- **Description:** CockroachDB provides full ACID compliance, ensuring reliable transaction processing. It supports complex transactions across distributed systems while maintaining strong consistency and isolation, similar to traditional SQL databases.



NoSQL Characteristics in NewSQL:

1. Horizontal Scalability:

- **Example: Google Spanner**
 - **Description:** Google Spanner achieves horizontal scalability by distributing data across multiple servers and data centers. This allows it to handle large volumes of data and high traffic loads while maintaining performance and availability.
- **Example: CockroachDB**
 - **Description:** CockroachDB also supports horizontal scaling, enabling it to grow by adding more nodes to the cluster. This distributed architecture allows CockroachDB to handle large datasets and high-throughput workloads efficiently.

2. Distributed Architecture:

- **Example: NuoDB**
 -  **Description:** NuoDB is designed with a distributed architecture that enables it to scale out across multiple nodes. It uses a distributed data model to handle high-velocity data and large-scale operations, providing fault tolerance and high availability.
- **Example: VoltDB**
 - **Description:** VoltDB employs a distributed, in-memory architecture that allows it to achieve high throughput and low latency. This architecture is optimized for real-time applications that require quick processing of large volumes of data.



Thanks!!